

JAVA × DATA STRUCTURES & ALGORITHMS

Java DSA Reference Handout

Companion notes for the 90-minute Java DSA training session — definitions, complexity tables, and ready-to-run Java code.

Track: Java Training | Duration: 90 minutes

CONTENTS

Table of Contents

1. Why DSA Matters
2. Big-O Notation
3. Arrays & Java Collections
4. Linked List
5. Stack & Queue
6. Recursion
7. Sorting Algorithms
8. Searching
9. Trees & Traversals
10. HashMap & HashSet
11. Graphs (BFS/DFS)
12. Problem-Solving Framework & Practice Problems

FOUNDATIONS

1. Why Data Structures & Algorithms?

- DSA is the shared vocabulary of efficient software — arrays, trees, graphs, and hashing show up in almost every real system.
- Picking the right structure can turn an $O(n^2)$ bottleneck into an $O(n \log n)$ or $O(n)$ solution.
- It's the backbone of technical interviews at most product companies.
- Java's `java.util` package gives a rich, ready-made toolbox — `ArrayList`, `LinkedList`, `HashMap`, `PriorityQueue` — to implement these ideas cleanly.

What good DSA skill buys you

- **Correctness** — solving the exact problem, including edge cases.
- **Efficiency** — time and space complexity you can reason about and defend.
- **Scalability** — code that stays fast as input size grows.

COMPLEXITY

2. Big-O Notation

Big-O describes how runtime or memory scales as input size (n) grows. It lets us compare algorithms independent of hardware or language quirks.

Notation	Name	Example
$O(1)$	Constant	Array index access
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Traversing an array/list
$O(n \log n)$	Linearithmic	Merge sort, Quick sort (avg)
$O(n^2)$	Quadratic	Bubble / Insertion sort
$O(2^n)$	Exponential	Naive recursive Fibonacci

Rule of thumb: drop constants and lower-order terms. $O(3n + 5)$ simplifies to $O(n)$. Always analyze the **worst case** unless stated otherwise.

DATA STRUCTURES

3. Arrays & Java Collections

Arrays

- Fixed-size, contiguous memory block — $O(1)$ index access.
- Declared as `int[] arr = new int[5];`
- Insertion/deletion in the middle is $O(n)$ since elements must shift.

```
int[] nums = {10, 20, 30, 40, 50};

// Access - O(1)
int third = nums[2];           // 30

// Traverse - O(n)
for (int n : nums) {
    System.out.print(n + " ");
}
```

The Collections Framework

Interface	Common Implementations	Notes
List	ArrayList, LinkedList	Ordered, index access, duplicates allowed
Set	HashSet, TreeSet	Unique elements only
Map	HashMap, TreeMap	Key → value pairs
Queue/Deque	ArrayDeque, PriorityQueue	FIFO / LIFO / priority order

DATA STRUCTURES

4. Linked List

- A chain of nodes; each node stores data plus a reference to the next node.
- No contiguous memory — insertion/deletion at the head is O(1).
- Traversal/search is O(n) — no random index access like arrays.
- Java's built-in LinkedList<T> implements both List and Deque.

```
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}

// Insert at head - O(1)
Node newHead = new Node(5);
newHead.next = head;
head = newHead;

// Traverse - O(n)
Node cur = head;
while (cur != null) {
    System.out.println(cur.data);
    cur = cur.next;
}
```

DATA STRUCTURES

5. Stack & Queue

Stack (LIFO)

- Last-In, First-Out — push() and pop() both act on the top.
- Prefer Deque<Integer> stack = new ArrayDeque<>(); over the legacy Stack class.
- Uses: undo/redo, expression evaluation, DFS, function call stack.

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
stack.push(30);

System.out.println(stack.pop()); // 30
System.out.println(stack.peek()); // 20
```

Queue (FIFO)

- First-In, First-Out — enqueue at the back, dequeue from the front.
- Use Queue<Integer> q = new ArrayDeque<>();
- Uses: task scheduling, BFS traversal, buffering.

```
Queue<Integer> q = new ArrayDeque<>();
q.offer(10);
q.offer(20);

System.out.println(q.poll()); // 10 (front)
System.out.println(q.peek()); // 20
```

TECHNIQUE

6. Recursion

- A method that calls itself, working toward a base case.
- Every recursive call needs: (1) a base case, (2) progress toward it.
- Backed by the call stack — deep recursion risks StackOverflowError.
- Powers divide-and-conquer algorithms: merge sort, quick sort, tree/graph traversal.

```
int factorial(int n) {
    if (n <= 1) return 1; // base case
    return n * factorial(n - 1); // recursive case
}

int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

ALGORITHMS

7. Sorting Algorithms

Algorithm	Average Time	Worst Time
-----------	--------------	------------

Bubble Sort	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n^2)$

Insertion Sort

```
void insertionSort(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

Merge Sort (Divide & Conquer)

Split the array in half recursively, then merge sorted halves back together. Stable, $O(n \log n)$, but uses $O(n)$ extra space.

```
void mergeSort(int[] a, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}
```

Quick Sort (Partition-Based)

Pick a pivot, partition so smaller elements go left and larger go right, then recurse on each side. In-place, average $O(n \log n)$, worst case $O(n^2)$ on a poor pivot choice.

```

void quickSort(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quickSort(a, lo, p - 1);
    quickSort(a, p + 1, hi);
}

int partition(int[] a, int lo, int hi) {
    int pivot = a[hi], i = lo - 1;
    for (int j = lo; j < hi; j++)
        if (a[j] < pivot) swap(a, ++i, j);
    swap(a, i + 1, hi);
    return i + 1;
}

```

ALGORITHMS

8. Searching

- **Linear Search** — check each element in order, $O(n)$, works on unsorted data.
- **Binary Search** — repeatedly halve the search range, $O(\log n)$, requires sorted data.
- Java shortcut: `Arrays.binarySearch(arr, target)`

```

int binarySearch(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        else if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

```

DATA STRUCTURES

9. Trees & Traversals

- Hierarchical structure: each node has up to 2 children (left, right) in a binary tree.
- BST property: left subtree < node < right subtree.
- Search/insert/delete average $O(\log n)$; $O(n)$ worst case if unbalanced.
- Balanced variants (AVL, Red-Black) guarantee $O(\log n)$ — Java's `TreeMap`/`TreeSet` use Red-Black trees internally.

```

class TreeNode {
    int val; TreeNode left, right;
    TreeNode(int v) { val = v; }
}

TreeNode insert(TreeNode n, int v) {
    if (n == null) return new TreeNode(v);
    if (v < n.val) n.left = insert(n.left, v);
    else n.right = insert(n.right, v);
    return n;
}

```

Traversal Orders

Traversal	Order	Typical Use
In-order	Left → Node → Right	Sorted order for a BST
Pre-order	Node → Left → Right	Copy / serialize a tree
Post-order	Left → Right → Node	Safely delete a tree

DATA STRUCTURES

10. HashMap & HashSet

- `HashMap<K,V>` stores key-value pairs with average $O(1)$ get/put via hashing.
- `HashSet<T>` stores unique values only — backed internally by a `HashMap`.
- Hash collisions are resolved via chaining (linked list / tree buckets since Java 8).
- Classic uses: frequency counting, deduplication, fast lookups, LRU caching with `LinkedHashMap`.

```

Map<String, Integer> freq = new HashMap<>();
for (String w : words) {
    freq.merge(w, 1, Integer::sum);
}

Set<Integer> seen = new HashSet<>();
if (!seen.add(num)) {
    // duplicate found - O(1) check
}

```

DATA STRUCTURES

11. Graphs — BFS & DFS

- A graph is a set of vertices connected by edges — models networks, maps, dependencies.
- Represent with an adjacency list: `Map<Integer, List<Integer>>`
- **BFS** (queue-based) explores level by level — finds shortest path in unweighted graphs.
- **DFS** (stack/recursion-based) explores as deep as possible before backtracking.
- Both run in $O(V + E)$ time — vertices plus edges.

```

void bfs(int start, Map<Integer, List<Integer>> g) {
    Queue<Integer> q = new ArrayDeque<>();
    Set<Integer> visited = new HashSet<>();
    q.offer(start); visited.add(start);
    while (!q.isEmpty()) {
        int node = q.poll();
        for (int nb : g.getOrDefault(node, List.of()))
            if (visited.add(nb)) q.offer(nb);
    }
}

```

PRACTICE

12. Problem-Solving Framework

- **Understand** — restate the problem, clarify inputs/outputs, edge cases.
- **Plan** — pick the right structure: array, map, stack, tree, graph?
- **Brute force** — get a correct, even if slow, solution first.
- **Optimize** — reduce time/space complexity, trade one for the other.
- **Test & trace** — walk through examples and edge cases by hand.

Worked example: Two Sum

Given an array, find two numbers that add up to a target value.

```

// Brute force - O(n^2)
for (int i = 0; i < nums.length; i++)
    for (int j = i + 1; j < nums.length; j++)
        if (nums[i] + nums[j] == target) return new int[]{i, j};

// Optimized with HashMap - O(n)
Map<Integer, Integer> seen = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
    int need = target - nums[i];
    if (seen.containsKey(need)) return new int[]{seen.get(need), i};
    seen.put(nums[i], i);
}

```

Practice problems by topic

- **Arrays/Strings:** Two Sum, Best Time to Buy/Sell Stock, Valid Anagram
- **Linked List:** Reverse a Linked List, Detect Cycle, Merge Two Sorted Lists
- **Stack/Queue:** Valid Parentheses, Min Stack, Implement Queue using Stacks
- **Trees:** Maximum Depth of Binary Tree, Validate BST, Level Order Traversal
- **HashMap:** Group Anagrams, First Unique Character, Subarray Sum Equals K
- **Graphs:** Number of Islands, Course Schedule, Clone Graph

Recommended resources

- docs.oracle.com/javase — official Collections API reference
- [GeeksforGeeks](#) — DSA in Java tutorials
- [LeetCode](#) — topic-wise practice with a Java-friendly editor
- “Cracking the Coding Interview” by Gayle Laakmann McDowell
- [NeetCode 150](#) roadmap

